OCPP-Server: Complete Project Documentation

Executive Summary

This project is a **full-stack EV Charging Station Management System** that enables remote monitoring and control of electric vehicle chargers using the industry-standard OCPP 1.6J protocol. It provides real-time dashboard visualization, intelligent load balancing with solar integration, and comprehensive session management.

Business Value

Benefit	Description
Cost Savings	Dynamic Load Balancing prevents expensive main fuse overloads and optimizes solar self-consumption
Remote Management	Control chargers from anywhere via web dashboard - no physical access needed
Data Analytics	Complete charging history with CSV export for billing, usage analysis, and reporting
Solar Optimization	PV Dynamic Balance mode maximizes free solar energy, reducing grid costs
Security	RFID whitelist ensures only authorized users can charge
Scalability	Supports unlimited chargers from a single server
Vendor Agnostic	Works with any OCPP 1.6 compliant charger (Z-Beny, ABB, Schneider, etc.)

System Architecture

```
┌──────────────
```

```
│
│   └──────────┘          │   │ • chargerService.js - State Management │    │
│
│        │                 │   │ • dlbService.js - Load Balancing       │    │
│
│        │                 │   │ • ocppHandlers.js - Protocol Handler   │    │
│
│   ┌────▼────┐            │   │ • apiRoutes.js - REST API              │    │
│
│   │  SOLAR  │            │   └────────────────────────────────────────┘    │
│
│   │ INVERTER │──────────►│                     │                          │
│
│   │ (CT/DLB) │ DLB Data │                     ▼                          │
│
│   └─────────┘            │             ┌──────────────┐                   │
│
│                          │             │   MongoDB    │                   │
│
│   ┌──────────┐           │             │ (Atlas/Local) │                  │
│
│   │ WEB BROWSER│◄────────►│             └──────────────┘                   │
│
│   │ (Dashboard)│   WS    │                                                │
│
│   └──────────┘  HTTP     └──────────────────────────────────────────────┘
│
│
│
└──────────────────────────────────────────────────────────────────────────
  ────┘
```

Complete Feature List

1. OCPP Protocol Implementation

Feature	Status	Description
BootNotification	☑	Charger registration and firmware detection
Heartbeat	☑	Keep-alive monitoring (30s interval)
MeterValues	☑	Real-time V/A/W/kWh readings (10s interval)
StartTransaction	☑	Session initiation with RFID auth
StopTransaction	☑	Session termination with energy recording
StatusNotification	☑	State changes (Available, Charging, Faulted)
RemoteStartTransaction	☑	Start charging from dashboard
RemoteStopTransaction	☑	Stop charging from dashboard

Feature	Status	Description
SetChargingProfile	☑	Dynamic power limit control
ChangeConfiguration	☑	Configure charger settings
GetConfiguration	☑	Read charger capabilities
Reset (Soft/Hard)	☑	Remote charger reboot
UpdateFirmware	☑	OTA firmware updates
DataTransfer	☑	Vendor-specific DLB data (Z-Beny)

2. Dynamic Load Balancing (DLB)

Mode	Description
PV Dynamic Balance	Charges only with solar surplus: `solarPower - homeLoad = chargerPower`
Extreme Mode	Full power charging, ignores all limits
Night Full Speed	Full power during off-peak hours (22:00-06:00)
Anti Overload	Prevents main fuse trip by throttling at 90% capacity

Energy Flow Visualization:

- Real-time hub-and-spoke diagram
- Bidirectional flow animation based on positive/negative power values
- Grid import/export, Solar generation, Charger consumption, Home load

3. Dashboard Features

Feature	Description
Device List	All connected chargers with status badges
Live Gauges	Animated power meter with wave fill effect
Statistics Cards	Energy (kWh), Power (W), Current (A), Voltage (V)
Charging Timer	Duration-based or schedule-based (start/end time)
History View	Monthly/yearly data with bar charts
CSV Export	Download charging sessions for billing
Fault Banner	Real-time fault detection with severity levels
Connection Status	Reconnection banner when WebSocket drops

4. Settings & Configuration

Setting	Description

Setting	Description
Power Limit	Per-charger max current (6-32A)
Main Fuse Rating	Configurable for Anti Overload calculations
RFID Whitelist	Authorized card management (persistent)
LED Brightness	Day/Night brightness with time schedule
Spot Tariff	Peak/Off-peak pricing configuration
Safety Toggles	Grounding detection, Emergency stop, Plug & Play
Firmware Upgrade	OTA updates from repository
Switch Mode	Disconnect OCPP for Bluetooth pairing

5. Data Persistence (MongoDB)

Collection	Purpose
`charging_sessions`	Complete transaction history
`charger_settings`	Per-charger configuration (RFID, limits, etc.)

6. Security Features

- RFID whitelist authorization
- Emergency stop system
- Grounding detection
- WebSocket connection validation
- Charger ID sanitization

File Structure

```
OCPP-Server/
├── server.js                    # Main entry - HTTP + WebSocket servers
├── package.json                 # Dependencies: ws, mongodb, dotenv
│
├── src/
│   ├── config/
│   │   └── config.js            # All constants (limits, timing, thresholds)
│   │
│   ├── constants/
│   │   └── ocppErrors.js        # OCPP error code mapping
│   │
│   ├── db/
│   │   └── database.js          # MongoDB connection & queries
│   │
│   ├── handlers/
│   │   ├── ocppHandlers.js      # OCPP message processing (1046 lines)
```

```
│   │   └── dashboardHandlers.js # Dashboard command processing
│   │
│   ├── routes/
│   │   └── apiRoutes.js         # HTTP REST API (880 lines)
│   │
│   ├── services/
│   │   ├── chargerService.js    # Charger state & session management
│   │   └── dlbService.js        # Load balancing algorithms
│   │
│   └── utils/
│       └── utils.js             # Utility functions (IST conversion, etc.)
│
├── views/
│   └── dashboard.html           # Single-page dashboard UI (1494 lines)
│
├── public/
│   ├── css/
│   │   └── dashboard.css        # All styling
│   └── js/
│       └── dashboard.js         # Frontend logic (2983 lines)
│
├── simulators/
│   └── charger-simulator.js     # Mock charger for testing
│
└── logs/
    └── session-*.log            # Auto-generated session logs
```

Technology Stack

Layer	Technology	Purpose
Runtime	Node.js 14+	Server-side JavaScript
Protocol	WebSocket (ws)	Real-time bidirectional communication
Database	MongoDB Atlas	Cloud-hosted persistent storage
Frontend	Vanilla JS + CSS	No framework dependencies
Fonts	Google Inter	Premium typography

Configuration Parameters

Parameter	Default	Description
`MIN_CHARGE_AMPS`	6A	Minimum EV charging current
`MAX_CHARGE_AMPS`	32A	Maximum hardware limit
`MAIN_FUSE_AMPS`	40A	Home main breaker rating
`NOMINAL_VOLTAGE`	230V	Phase voltage

Parameter	Default	Description
`PHASES`	3	Three-phase supply
`NIGHT_START_HOUR`	22	Night mode start
`NIGHT_END_HOUR`	6	Night mode end
`METER_INTERVAL`	10000ms	Meter value reporting interval
`HEARTBEAT_INTERVAL`	30000ms	Keep-alive interval
`DLB_SYNC_INTERVAL`	15000ms	Load balancing sync interval

Deployment Requirements

Requirement	Specification
Node.js	>= 14.0.0
MongoDB	Local or Atlas (connection string in .env)
Port	9000 (configurable)
Network	Chargers must reach server IP on port 9000

How It Works - Charging Session Flow

```
1. Charger connects → WebSocket /ocpp/CHARGER_ID
2. Charger sends BootNotification → Server accepts, loads saved settings
3. User plugs in EV → Charger sends StatusNotification: Preparing
4. User taps RFID → Charger sends Authorize → Server checks whitelist
5. Dashboard clicks Start → Server sends RemoteStartTransaction
6. Charging begins → Charger sends MeterValues every 10 seconds
7. DLB calculates optimal power → Server sends SetChargingProfile
8. Dashboard clicks Stop → Server sends SetChargingProfile(0W), then Reset
9. Session saved to MongoDB with energy, duration, timestamps
10. History view shows session data, exportable as CSV
```

Supported Charger Brands

Successfully tested with:

- **Z-Beny** (EV-Box, DLB receiver support)
- **EVB/NewSolar** (Power.Grid, Power.Solar format)
- **Any OCPP 1.6J compliant charger**

API Endpoints

Charger Management

- `GET /api/chargers/all` - List all connected chargers
- `POST /api/charger/disconnect` - Disconnect charger (switch to Bluetooth)

Dynamic Load Balancing

- `GET /api/dlb/status` - Current DLB state
- `POST /api/dlb/config` - Update DLB mode settings

Settings

- `GET /api/settings/power-limit` - Get charger power limit
- `POST /api/settings/power-limit` - Set charger power limit
- `POST /api/settings/rfid/add` - Add RFID tag to whitelist
- `POST /api/settings/rfid/remove` - Remove RFID tag from whitelist

History & Analytics

- `GET /api/history` - Get charging sessions
- `GET /api/history/chargers` - List chargers with history
- `GET /api/history/download` - Export sessions as CSV

Firmware

- `POST /api/firmware/update` - Trigger OTA firmware update

Unique Selling Points

1. **Solar-First Charging** - Only project with true PV Dynamic Balance using real solar data
2. **Visual Energy Flow** - Hub-and-spoke diagram with animated dots showing real-time power direction
3. **Comprehensive DLB** - Four intelligent modes for different use cases
4. **Single Codebase** - No external frameworks, easy to deploy and maintain
5. **Full Audit Trail** - Session logging for compliance and billing
6. **Mobile-Optimized** - Responsive design works on phones and tablets

---

Project Statistics

Metric	Value
Total JavaScript Lines	~5,500+
HTML Lines	~1,500
CSS Lines	~1,000+
API Endpoints	25+
OCPP Messages Handled	15+

Metric	Value
External Dependencies	Only 3 (ws, mongodb, dotenv)

Getting Started

Installation

```
# Clone repository
git clone https://github.com/SivaSajee/ocpp-server.git
cd ocpp-server

# Install dependencies
npm install

# Configure environment
cp .env.example .env
# Edit .env with your MongoDB URI

# Start server
npm start
```

Accessing Dashboard

Open browser: `http://localhost:9000`

Connecting a Charger

Configure charger OCPP endpoint: `ws://YOUR_SERVER_IP:9000/ocpp`

Testing with Simulator

```
npm run simulator
```

Document generated: March 5, 2026